Ask or Act for Cooperative Assistance via Inverse Planning

CS6208 Final Presentation

Zhu Yufan
A0238993J

April 2026

Problem

- A cooperative helper must choose between acting now and asking a clarifying question.
- Under ambiguous language and a hidden goal, acting too early risks the wrong object.
- Asking too often also has cost: it consumes time and can delay task completion.

Goal

Learn a policy that asks **only when the expected value of information outweighs the cost**.

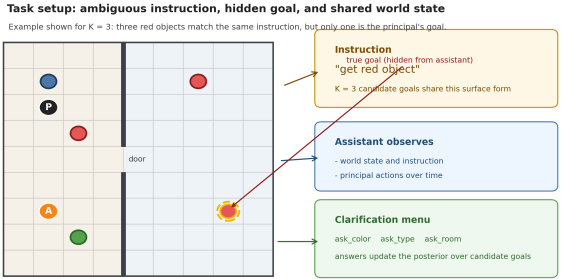
Core question

When should a helper seek more information instead of committing to a likely goal?

- Hidden principal goal
- Ambiguous instruction
- Noisy observed behavior

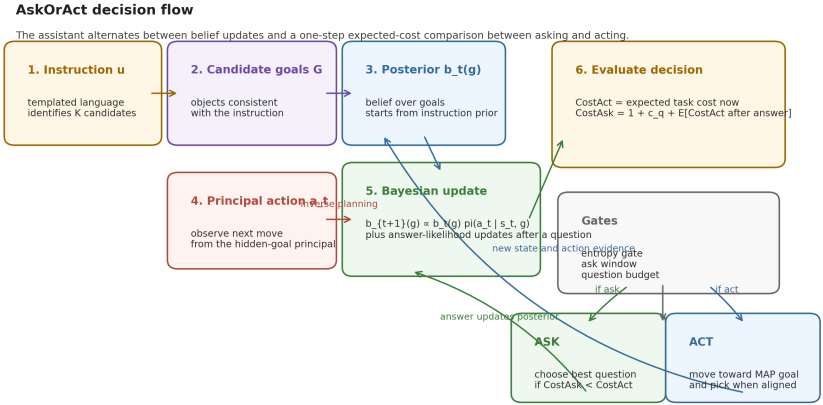
Task Setup

- 9×9 two-room gridworld with 6 objects.
- Principal has a **hidden goal object** and gives an ambiguous instruction.
- Ambiguity level $K \in \{1, 2, 3, 4\}$ is the number of candidate goals matching the instruction.
- Principal acts approximately rationally with Boltzmann parameters β and ε.
- Assistant sees state, instruction, and principal motion, but not the true goal.



AskOrAct Method

- Maintain a posterior over candidate goals from the instruction and inverse planning.
- Update the posterior after each observed principal action.
- Compare the expected team cost of acting now against asking the best available question.



Decision Rule and Baselines

Ask-or-act comparison

$$\text{CostAct} = \sum_g b_t(g)\,\text{dist}(s_t, g)$$

$$\text{CostAsk}(q) = 1 + 0.5 + \mathbb{E}_\alpha[\text{CostAct after answer } \alpha]$$

Ask the best question only if it beats acting now.

- Entropy gate: skip asking if already confident
- Ask window: no new questions after step 6
- Budget: at most 3 questions per episode

- **AskOrAct**: ask only when VoI exceeds cost
- **NeverAsk**: act on MAP goal
- **AlwaysAsk**: ask until budget
- **InfoGainAsk**: gated highest-IG ask
- **EasyInfoGainAsk**: one-question IG
- **RandomAsk**: gated random ask
- **POMCPPlanner**: POMDP search at K=2

Evaluation Protocol

- Sweep: $K \in \{1, 2, 3, 4\}$, $\varepsilon \in \{0, 0.05, 0.1\}$, $\beta \in \{1, 2, 4\}$.
- 5 replicate seeds and 20 episodes per condition.
- 5 core policies evaluated in every cell.
- EasyInfoGainAsk and POMCPPlanner added as targeted stronger baselines at $K{=}2$.

Metrics

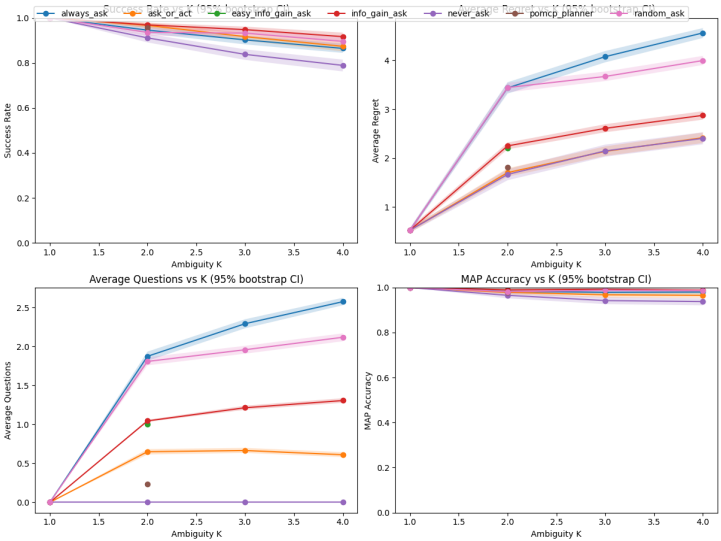
Success rate, regret vs. oracle, questions per episode, and final MAP accuracy with 95% bootstrap confidence intervals.

Scale

19,800 episodes

- Reproducible seeds
- Dynamic deadlines
- Assistant-only completion

Main Results



- K=1: all policies collapse to the same easy case.
- K=3: AskOrAct beats NeverAsk by **+7.9 pp** success.
- K=4: AskOrAct beats NeverAsk by **+8.4 pp** success.

Efficiency

AskOrAct reaches those gains while asking only about **0.6 questions per episode**.

Over-Asking vs Under-Asking

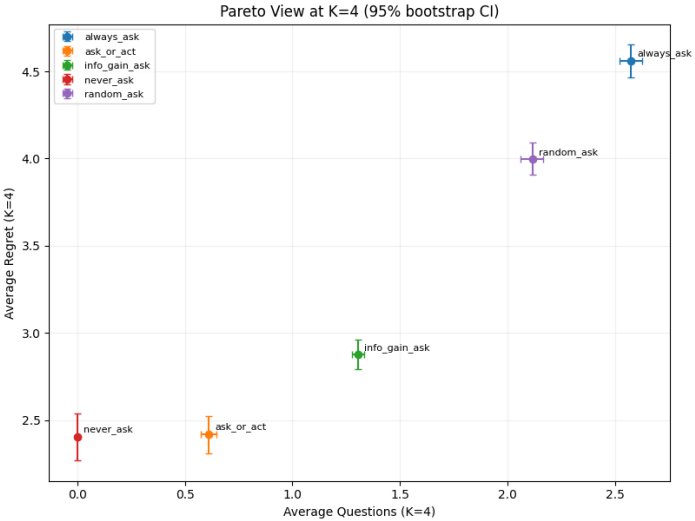
Policy	Success	Regret	Q/ep
InfoGainAsk	**91.7%**	2.88	1.31
AskOrAct	87.3%	**2.42**	0.61
AlwaysAsk	86.4%	4.56	2.58
NeverAsk	78.9%	2.40	0.00

Interpretation at $K=4$

AskOrAct is the best balance point: it avoids NeverAsk's commitment errors and avoids AlwaysAsk's communication overhead.

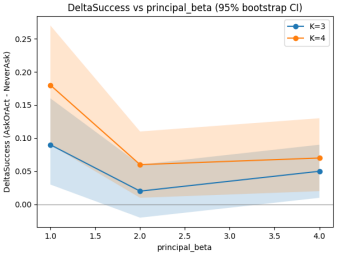
- AlwaysAsk uses about **4x** as many questions as AskOrAct.
- That extra questioning does **not** translate into better team cost.
- InfoGainAsk gets higher success, but pays for it in regret.

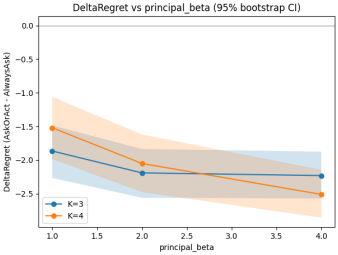
Pareto View



- AskOrAct is **Pareto-efficient** at high ambiguity.
- It dominates NeverAsk on success.
- It dominates AlwaysAsk and RandomAsk on regret.
- InfoGainAsk reaches higher success, but only by paying a higher coordination cost.

Robustness and Ablations





- Under principal-model mismatch, AskOrAct still outperforms NeverAsk.
- Hardest case: true $\beta = 1$ gives about **+13.5 pp** success over NeverAsk.
- Answer-noise sweeps and scale-K tests preserve the same ordering.

Ablation takeaway

A non-zero question cost is essential. Zero-cost asking leads to over-asking and lower success.

Takeaways

1. **Selective asking works.** AskOrAct improves success by about 8 pp at $K=3,4$ with fewer than 1 question per episode.
2. **Both extremes fail.** NeverAsk under-queries; AlwaysAsk over-queries and pays high regret.
3. **Communication cost matters.** Without it, the policy over-asks and becomes less effective.
4. **The pattern is robust.** The advantage survives answer noise, principal mismatch, and held-out template tests.

Next step

Extend the framework beyond templated questions and small exact-inference environments.

Backup: Evaluation Bug Found and Fixed

- An early setting used `ENTROPY_GATE = 0.8`, which prevented AskOrAct from asking at $K{=}2$.
- Reason: the uniform prior over two goals has entropy $\ln 2 \approx 0.69$, so the gate always fired.
- Fix: reduce the threshold to `0.3`.
- After the fix, AskOrAct at $K{=}2$ improves to **96.7%** success vs. NeverAsk at **91.1%**.

Why keep this slide?

Careful debugging and consistency checks were part of the empirical contribution, not separate from it.